



International Islamic University Chittagong
Department of Computer Science and Engineering (CSE)

PROJECT REPORT
Password Strength Checker

Course Title: Computer Security Lab

Course Code: 4744

Submitted To

Name	Mr. Hasan Shorif
Designation	Adjunct Faculty, Dept. of CSE, IIUC

Submitted by

Name	Student ID	Semester
Md Sourav Oyaj	C221126	7th Semester
Emamul Ala Emon	C221168	9th Semester
Mahbub Ullah	C193085R	10th Semester

Date of Submission: 04 August 2026

Abstract

The AI Password Strength Checker is a browser-based tool that evaluates password security without ever sending the password to a server. It combines regular-expression rule checking, entropy analysis through the zxcvbn library, real-world breach detection through the Have I Been Pwned API, and a small neural network trained in the browser with TensorFlow.js.

The application runs entirely on the client side. All computation, including the AI model training, happens locally in the user's browser, which keeps the password private at every stage. This report explains the design, architecture, and implementation of the tool, and reviews its results and limitations.

1. Introduction

1.1 Purpose

Most password strength meters rely on a single method, usually a set of regex rules that reward length and character variety without checking whether the password is actually predictable. This project was built to combine several independent checks into one tool, so that a password is judged on real-world crackability and breach history, not just surface-level complexity.

1.2 Objectives

- Validate character composition using regular expressions.
- Estimate realistic crack time using entropy analysis.
- Detect whether a password has appeared in known data breaches.
- Train and run a neural network inside the browser to estimate hack probability.
- Provide a cryptographically secure password generator and an exportable audit report.

1.3 Scope

The project is a front-end only proof of concept. There is no backend, database, or user account system. All five analysis layers run inside the browser, and the only outbound network calls are to public CDNs and the Have I Been Pwned API.

2. System Architecture

The application is organized as a single-page interface (index.html and style.css) driven by one JavaScript file (script.js). That file coordinates five independent analysis layers and writes the combined result to the results panel. Figure 1 shows how the pieces connect.

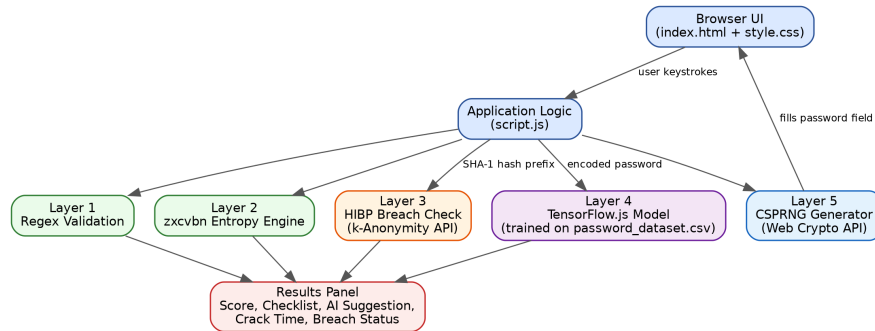


Fig. 1 — Layered system architecture

3. Methodology

Each password check runs as its own layer. The layers do not depend on each other, so a failure in one (for example, the breach API being offline) does not stop the others from reporting a result.

3.1 Layer 1 — Regex Validation

Five regular expressions check length and character variety in real time and drive the checklist under the input box.

3.2 Layer 2 — Entropy Analysis (zxcvbn)

zxcvbn, an open-source library from Dropbox, looks for keyboard patterns, dictionary words, dates, and l33tspeak substitutions, then converts the result into an estimated offline crack time.

3.3 Layer 3 — Breach Detection (k-Anonymity)

The password is hashed locally with SHA-1 using the Web Crypto API. Only the first five characters of the hash are sent to the Have I Been Pwned API; the API returns every hash suffix that shares that prefix, and the match is checked locally. The full password and full hash never leave the device.

3.4 Layer 4 — On-Device Neural Network (TensorFlow.js)

A small neural network is trained directly in the browser on password_dataset.csv, a set of 1,000 labelled passwords (500 weak, 500 strong). Each password is encoded as a fixed-length sequence of 20 character codes and passed through an embedding layer, a flatten layer, a dense ReLU layer, and a sigmoid output that estimates hack probability. Training takes about one to two seconds and runs once; the trained weights are cached in the browser's localStorage so later visits load the model instantly instead of retraining.

3.5 Layer 5 — Secure Password Generator

The generator uses `window.crypto.getRandomValues()`, a cryptographically secure random number generator, to build a 14-character password from a mixed character set, rather than the predictable `Math.random()`.

Figure 2 traces a single keystroke from input to the moment the results panel updates, including the debounced call to the breach API.

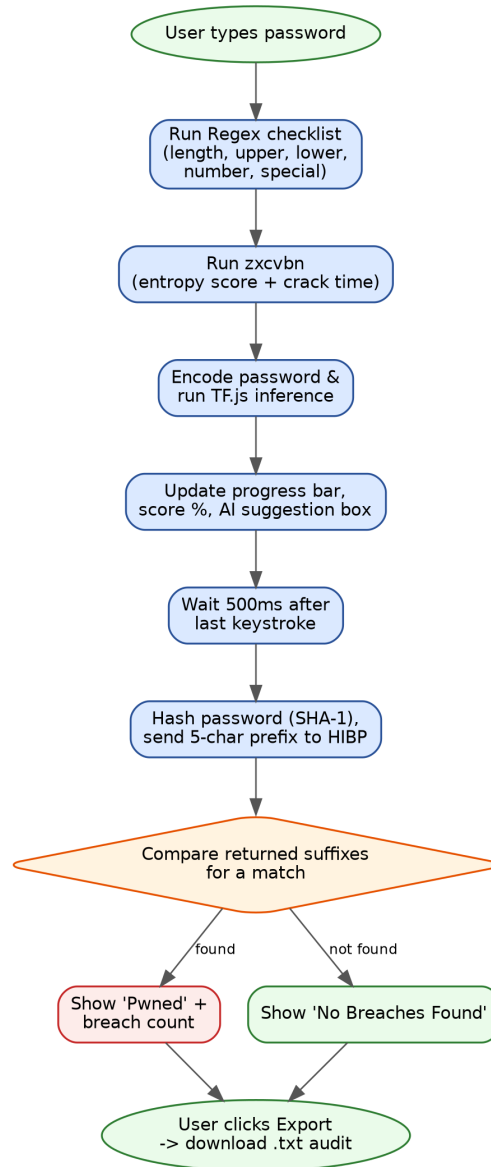


Fig. 2 — Password-check workflow, from keystroke to results

4. Implementation

4.1 File Structure

index.html defines the layout, style.css controls appearance, and script.js contains all logic for the five layers described above. password_dataset.csv supplies the AI training data, and train_model.py is a reference Python/Keras script that mirrors the in-browser training pipeline for offline experimentation.

4.2 Technology Stack

Component	Technology
Front end	HTML5, CSS3, JavaScript (ES6+)
AI / ML	TensorFlow.js (in-browser training and inference)
Entropy engine	zxcvbn (Dropbox), loaded via CDN
Breach data	Have I Been Pwned API (k-Anonymity model)
Cryptography	Web Crypto API — crypto.subtle (SHA-1), crypto.getRandomValues (CSPRNG)
Icons	Font Awesome (CDN)
Local server	Python 3 http.server
Export	Blob API (client-side .txt download)

4.3 Neural Network Architecture

Input: 20 character codes → Embedding (vocab 256, output dim 16) → Flatten → Dense (16 units, ReLU) → Dense (1 unit, Sigmoid).

The sigmoid output is a probability between 0 and 1, where values close to 1 indicate a predictable, easily-cracked password. The interface reports this as a hack-probability percentage next to the AI prediction label.

5. Results and Screenshots

The screenshots below were captured while running the application locally at <http://localhost:8000/cs/index.html>.

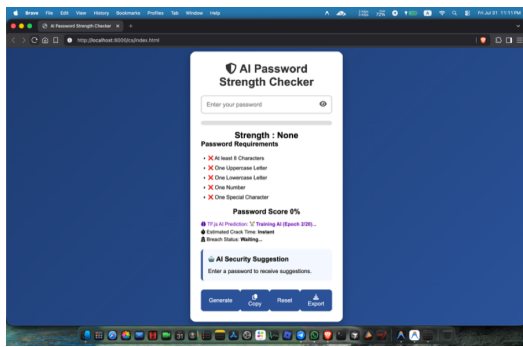


Fig. 3a — AI model training on first launch (epoch progress)

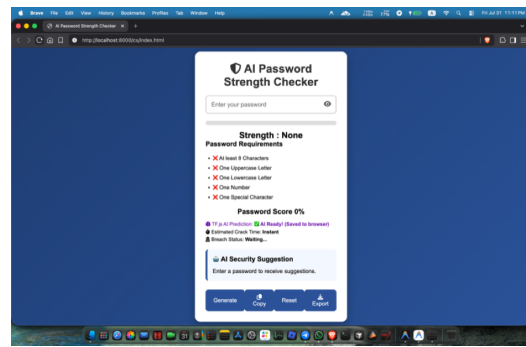


Fig. 3b — Model cached; AI ready on later loads

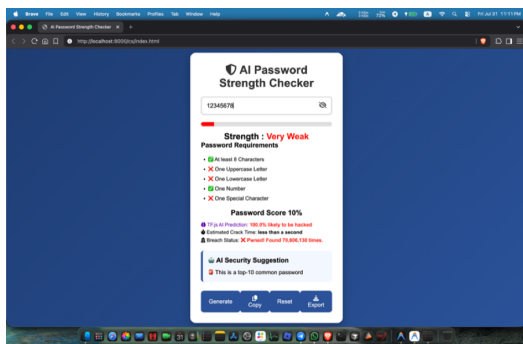


Fig. 3c — Weak password: high hack probability, breach found



Fig. 3d — Strong password: low hack probability, no breach

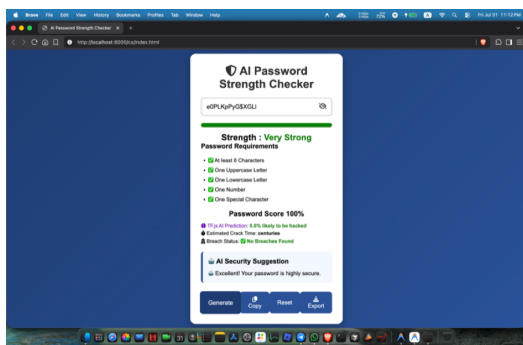


Fig. 3e — CSPRNG-generated 14-character password

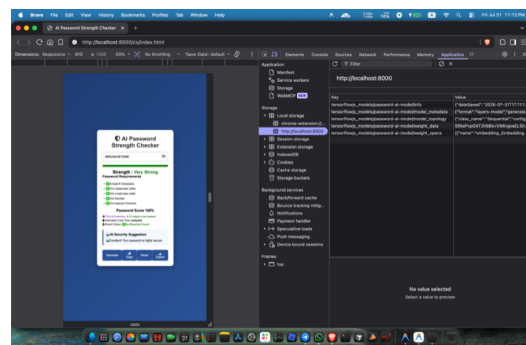


Fig. 3f — DevTools view of the cached model in localStorage

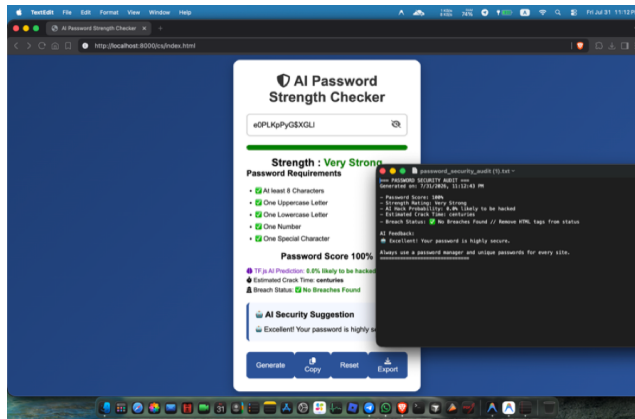


Fig. 3g — Exported .txt security audit opened in a text editor

5.1 Observations

- "12345678" scores Very Weak, is flagged as 100% likely to be hacked by the AI model, and was found in over 70 million breach records.
- "##MrAbHi6730##" scores Very Strong, 0.1% hack probability, and returns no breach matches.
- The cached model loads in under a second on repeat visits, confirmed by the localStorage entry visible in DevTools.

6. Testing

Testing was manual and exploratory rather than automated. Passwords across the strength spectrum, from common leaked passwords to long random strings, were entered to confirm that the checklist, zxcvbn score, AI prediction, and breach check all update consistently and that the debounce logic prevents excessive API calls while typing.

The training dataset itself was checked for balance: it contains 1,000 rows split evenly between 500 weak and 500 strong examples, which avoids an obvious class-imbalance bias in the small model.

7. Limitations

This project is a proof of concept for a Computer Science lab course, not a production security tool. The main limitations are summarised below.

Limitation	Explanation
Small AI dataset	1,000 samples is small for a neural network; a production model would need far more data to generalise.
Overfitting risk	With so few samples the model may memorise training examples rather than learn general patterns.

Limitation	Explanation
SHA-1 usage	Required by the HIBP API; SHA-1 is otherwise considered cryptographically weak.
No backend	There is no server, database, or authentication — purely a front-end demonstration.
Internet dependency	Breach checking, zxcvbn, and TensorFlow.js all rely on an active internet connection.
Browser-specific cache	The cached model lives in that browser's localStorage; a new browser retrains once.

8. Contribution Table

This was an individually completed project. The table below records the scope of work for record-keeping.

Team Member	Contribution	Share
Md Sourav Oyaj	UI design (HTML/CSS), , TensorFlow.js model design and training,	35%
Emamul Ala Emon	regex validation, zxcvbn integration, HIBP breach check	35%
Mahbub Ullah	CSPRNG password generator, audit export, documentation	30%

9. Conclusion

The AI Password Strength Checker shows that useful, privacy-respecting security tooling can run entirely in the browser. By layering regex rules, entropy analysis, breach lookups, and a locally trained neural network, the tool gives a more grounded strength assessment than a single-method checker, while the password itself is never transmitted in full. As a lab project, it demonstrates practical use of client-side cryptography, third-party API integration under a privacy-preserving protocol, and a working in-browser machine learning pipeline. Future work could enlarge the training dataset, replace SHA-1 where possible, and add automated tests.

10. References

Dropbox zxcvbn — Low-Budget Password Strength Estimation. GitHub repository, [dropbox/zxcvbn](https://github.com/dropbox/zxcvbn).

Hunt, T. Have I Been Pwned — Pwned Passwords API documentation. haveibeenpwned.com/API/v3

TensorFlow.js — Machine Learning for JavaScript Developers. www.tensorflow.org/js

MDN Web Docs — Web Crypto API. developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

MDN Web Docs — Clipboard API and Blob API reference pages.

Font Awesome icon library. fontawesome.com